

@SAPTARISHI

cds-plugin-llm + cds-plugin-vector-hana

Two CAP-native plugins that turn SAP Joule projects into production-grade AI platforms — in one line of code.

cds-plugin-llm 2.38.0 · cds-plugin-vector-hana 0.13.0

Every SAP LLM project rebuilds the same 40 layers

- ▶ The demo works. Then finance asks for a budget cap. Ops asks for retries. Legal asks for PII scrubbing. Security asks for prompt-injection defense.
- ▶ Every team hand-builds cost / resilience / security / observability from scratch. Six months later there's still no A/B framework, no rate-limit retry with jitter, no cross-provider fallback, no cost forecast.
- ▶ SAP's own Generative AI Hub gives you a model endpoint. It does NOT give you the 40 middleware layers between the endpoint and a production POS.
- ▶ Meanwhile CAP is the SAP-native way to build services. What if the AI plumbing came in the same shape as any other CAP dependency?

Two plugins. One line each.

cds-plugin-llm

11 providers behind one API

- ▶ Anthropic · Google Gemini · Bedrock · Azure OpenAI
- ▶ Ollama · Groq · Fireworks · DeepSeek · Mistral
- ▶ OpenAI-compatible · SAP Generative AI Hub
- ▶ 40+ middleware primitives · 2.x API stability contract
- ▶ Koa-style llm.use() composition

cds-plugin-vector-hana

@rag annotation on any CAP entity

- ▶ HANA Cloud REAL_VECTOR + SQLite dev fallback
- ▶ Hybrid vector + keyword (RRF fusion)
- ▶ HyDE query expansion, LLM rerank
- ▶ Auto-declares searchByMeaning / askAbout OData actions
- ▶ Zero handler code — pure declarative RAG

cds-plugin-llm — 40+ primitives across 10 categories

Cost

budget · guard · forecast · quota · aware routing · overrun predictor

Security

PII redact · reversible tokens · injection guard · guardrails · HMAC signing

Routing

model · tenant · cost-aware · fair-share · semantic · load balancer

Contract

JSON schema validate · repair · function-call arbitrator · length gate

RAG + Eval

ragChain · llmJudge · promptRegression · consensusVoting · A/B experiment

Resilience

circuit breaker · bulkhead · deadline · retry · region failover · hedge

Observability

OTel · Prometheus · JSON log · health probe · latency histograms · MCP

Caching

exact · semantic · embedding dedup · request coalescer · fuzzy dedup

Long-context

compact history · session store · reversible tokenization

Multi-agent

Agent · runAgents · streamAgents · autoToolChain · supervisor

Cost primitives — turn AI spend into an SLO

- ▶ costGuard — pre-flight per-call USD ceiling (rejects before the API call)
- ▶ costBudget — per-tenant / per-model / per-window ceilings; hydrate from a CAP entity, edit live, no restart
- ▶ costForecast — rolling-window spend projection with rising-edge alerts ("you'll hit the \$50 target at 2:47pm")
- ▶ quotaManager — per-user USD quota with warnings before the ceiling
- ▶ costOverrunPredictor — calendar-window burn-rate + startOfMonth / endOfMonth helpers
- ▶ costAwareRouter — cheap model first, escalate to expensive on schema failure
- ▶ adaptiveMaxTokens — shrink maxTokens automatically to fit remaining budget

WHY IT MATTERS

The finance conversation flips.

Instead of "what did this cost last month?" (reactive), finance now says "here's the ceiling per tenant per hour" and the plumbing enforces it. Overspend becomes a structurally impossible failure mode, not a monitoring problem.

Resilience + Security — production defaults, not homework

Resilience

- ▶ `retryOnRateLimit` — exponential backoff, honors `Retry-After`
- ▶ `circuitBreaker` — per-provider open/half-open/closed
- ▶ `bulkhead` + `adaptiveBulkhead` — AIMD tuning on p95 latency
- ▶ `deadline` — hard per-call time budget
- ▶ `gracePeriod` — soft deadline warnings + optional hard timeout
- ▶ `speculativeHedge` — staggered parallel hedges for tail latency
- ▶ `regionFailover` — `chatWithFallback` across N regions
- ▶ `chaosInjector` — deterministic seeded fault injection (test-only)

Security

- ▶ `guardrails` — regex + PII + blocklist stages
- ▶ `promptInjectionGuard` — 6-detector jailbreak defense
- ▶ `piiRedact` — round-trip PII masking with reversible tokens
- ▶ `safetyClassifier` — moderation + Anthropic refusal detection
- ▶ `requestSigning` — HMAC receipts + `verifyReceiptChain`
- ▶ `responseSigning` — HMAC responses (tamper-evident audit trail)
- ▶ `sensitiveDataAudit` — before/after diff capture
- ▶ `emptyResponseDetector` — refusal-pattern detection + auto-retry

Observability — every middleware is a live MCP resource

- ▶ OpenTelemetry spans with cost + correlation + routing + errors
- ▶ Prometheus /metrics text-exposition — one endpoint, all counters
- ▶ JSON structured logs (one line per call) with error taxonomy
- ▶ providerHealthProbe + healthAggregate — unified verdict per provider
- ▶ latencyHistogram — per-dimension p50/p95/p99 with Prom export
- ▶ traceCorrelation via uuidv7 (sortable, deterministic)
- ▶ replayBuffer — rolling in-memory window for on-call debugging
- ▶ usageMeteringToCap — auto-persists to a CAP entity (queryable via OData)

MCP-FIRST

Every counter is a subscribable resource.

`asMcpResource()` ships on every middleware. Claude Desktop, Cline, Cursor can subscribe to `config://budget`, `config://circuit-breaker`, `config://fuzzy-dedup` — and get a push notification the moment a counter changes.

Fuzzy Dedup — the third layer in the dedup story

- ▶ Layer 1: requestCoalescer — byte-identical, in-flight
- ▶ Layer 2: responseCache — byte-identical, at-rest
- ▶ Layer 3: semanticCache — embedding-similar (needs embedder)
- ▶ Layer 4 (NEW): fuzzyDedup — character-similar (no embedder)

Jaccard-trigram or normalized Levenshtein similarity. Catches supplier IDs retyped with dashes vs spaces, whitespace-only diffs, single-character typos, or extra optional fields — BEFORE the semantic cache spends an embedding round-trip.

LIVE ON THE PROCUREMENT COPILOT

3 near-duplicate POST /ai/summarizePurchaseOrder calls

totalCalls	3
fuzzyHits	2
hitRate	0.667
lastSimilarity	0.834
LLM calls saved	2 of 3

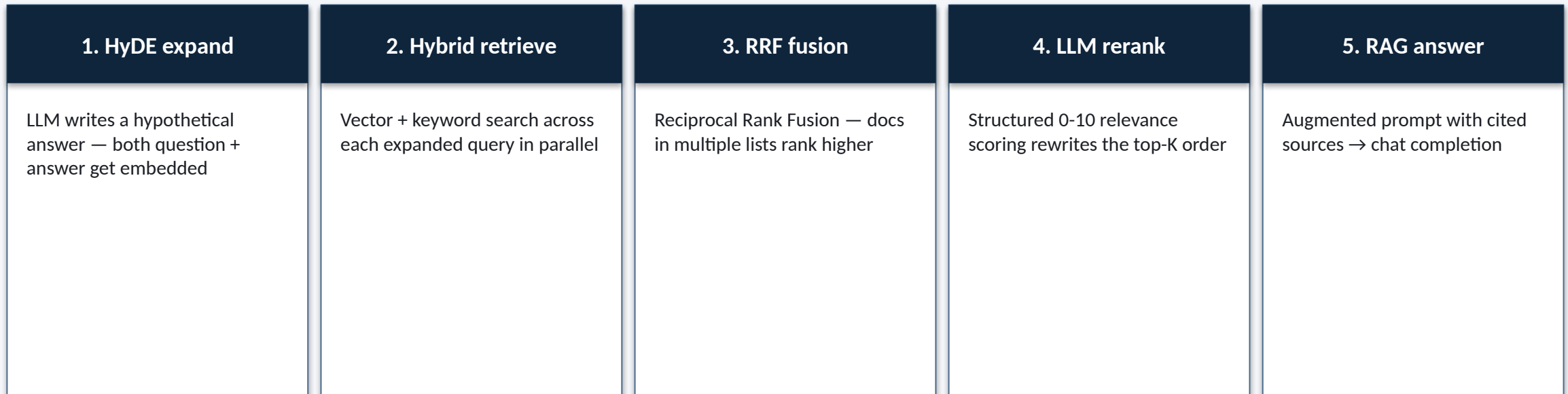
cds-plugin-vector-hana — @rag on any CAP entity

- ▶ One annotation on your existing CAP entity — no boilerplate handlers
- ▶ HANA Cloud REAL_VECTOR + COSINE_SIMILARITY in production
- ▶ SQLite fallback for local dev — no HANA needed to build features
- ▶ Hybrid search (vector + keyword via FTS5 / CONTAINS) with alpha knob
- ▶ Metadata filters: filter: { region: 'EMEA' } exact-match on any field
- ▶ Auto-declares searchByMeaning + askAbout OData actions on the entity
- ▶ Composes with cds-plugin-llm for embeddings + rerank + answer

CDS FILE

```
entity SupplierContracts {
  key ID    : String;
  supplier : String;
  region   : String;
  terms    : LargeString;
} @rag: {
  fields:
  ['supplier', 'region', 'terms'],
  metadataFields: ['region', 'supplier'],
  dimension:      768,
  store:          'hana',
  search:         'hybrid',
};
```

5-stage RAG pipeline — every stage a separate primitive



Groq Llama-3.3-70B end-to-end: ~2.5s. Every stage is individually swappable (custom rerankers, alternate expanders, different fusion strategies) — this is the composed default.

Joule Procurement Copilot — real customer-ready proof

- ▶ A live SAP Joule agent using both plugins end-to-end
- ▶ Actions: summarize PO · explain invoice risk · extract line items · multi-agent scenario analysis · voice memo → PO draft · batch supplier scoring
- ▶ 29-layer middleware chain (OTel → fuzzy dedup → semantic cache → provider)
- ▶ Auto-declared RAG actions on SupplierContracts entity (hybrid + HyDE)
- ▶ Deployed live on Render for the demo call — public URL, click and demo
- ▶ MCP observability surface on port 3334 — subscribe to any middleware's counters

29

middleware layers

11

LLM providers

40+

primitives shipped

~2.5s

5-stage RAG p95

0.667

fuzzy hit rate (demo)

Getting started — one npm install, one line of code

TERMINAL

```
$ npm install @saptarishi/cds-plugin-llm \
               @saptarishi/cds-plugin-vector-hana

// srv/ai-service.js
const llm = await cds.connect.to('llm');

llm.use(costBudget({ perTenant: { acme: 100 } }));
llm.use(promptInjectionGuard());
llm.use(piiRedact());
llm.use(fuzzyDedup({ store: inMemoryFuzzyStore() }));
llm.use(responseCache({ semantic: { embedder } }));

const { text } = await llm.chat({
  messages: [{ role: 'user', content: 'summarise P0-4471' }],
});
```

RESOURCES

npm

[@saptarishi/cds-plugin-llm@2.38.0](#)

[@saptarishi/cds-plugin-vector-hana@0.13.0](#)

Source

[github.com/kalyanjanumpally/
sap-joule-ai-plugin](https://github.com/kalyanjanumpally/sap-joule-ai-plugin)

Live demo

Procurement Copilot on Render (URL provided during the call)

Contact

jkalyan@alumni.iitm.ac.in